STC: Reversible Digit-Context Decomposition for BWT-Family Text Compression

Jingyang Du[1], Yang Shen[2,3], Anling Xiang[1], Yucheng Liu[4]

[1]School of Journalism and Communication, Minzu University of China, Beijing, China

[2]School of Journalism and Communication, Tsinghua University, Beijing, China

[3]College of AI, Tsinghua University, Beijing, China

[4]Department of Engineering Science, University of Oxford, Oxford, United Kingdom

[*]Corresponding author: Yang Shen (124739259@qq.com)

ORCID: Anling Xiang 0000-0003-1690-1586; Yang Shen 0000-0003-4814-9018

Abstract

Burrows–Wheeler-transform (BWT) compressors exploit repeated textual context, while structured text also contains digit runs whose values vary differently from their surroundings. We present STC, a reversible digit-context decomposition discovered by the authors through the self-evolving AI system ZiYor. STC replaces digit bytes in the main stream by an unambiguous placeholder, moves the removed payloads to run-length buckets, and orders bucket records by context reconstructible from the normalized main stream. Stable ordering lets the decoder reproduce the record permutation without storing it. With a fixed BWT/M03-style backend on enwik9, full STC produces a 157,388,188-byte archive, 2,629,561 bytes smaller than the same coder without digit decomposition. Including a 183,174-byte source-contained decoder package gives a reproducible source-accounted total of 157,571,362 bytes. As of September 27, 2026, the independently maintained LTCB lists STC at rank 37 overall and as the highest-ranked entry classified as BWT. Full decoding reproduces the 1,000,000,000-byte input exactly.

1 Introduction

The Burrows–Wheeler transform (BWT) reorganizes a block so that symbols with similar contexts tend to cluster, enabling effective downstream coding [1, 2]. Practical BWT compressors therefore depend not only on the transform but also on how source structure is exposed to it [3, 4]. Structured text such as Wikipedia XML mixes prose and markup with dates, identifiers, counters, coordinates, and other digit runs. The textual neighborhoods of these numbers may repeat even when the numeric payloads do not.

STC separates these two sources of variation before the component BWT stage. Every ASCII digit is replaced by one placeholder in a normalized main stream. The re-

moved digit runs are assigned to side streams by length, packed compactly, and stably ordered by local context computed from the normalized main stream. After decoding the components, the decoder can rediscover every run position, length, bucket, and context key from that same main stream; hence the side-stream permutation need not be serialized.

STC was discovered through the authors' self-evolving AI system ZiYor, which explored and refined candidate reversible compression transforms. This paper evaluates the resulting compression algorithm; a general evaluation of ZiYor or recursive self-improvement is outside its scope.

The main contributions are:

1. Introduced STC, discovered through ZiYor, as a schema-free reversible digit-context decomposition for BWT-family text compression.
2. Designed length/context bucket side streams with decoder-reconstructible stable ordering and compact digit packing.
3. Demonstrated a 2,629,561-byte (1.64%) archive reduction in a same-coder full-enwik9 ablation.
4. Provided reproducible benchmark accounting, source-contained decoding, and full-decode verification.

2 Related Work

BWT compression and its engineering have been studied extensively [1, 2, 3, 4, 5]; practical construction work likewise emphasizes that backend organization matters [6], while bsc-m03 provides a useful BWT-family reference point [7]. Reversible preprocessing similarly seeks to expose regularity before a backend coder, including dictionary/text transforms and XML-oriented compressors [8, 9, 10, 11, 12]. Columnar systems likewise show that numeric or typed fields can benefit from representations different from surrounding text [13, 14, 15, 16].

STC lies between these lines of work. Unlike XML-oriented preprocessors, it does not require a grammar, element model, or serialized structural map; unlike columnar and integer codecs, it does not assume that numeric values have already been isolated into typed fields. Maximal ASCII digit runs are identified syntactically at byte level, while the surrounding prose or markup remains in the main text stream.

Its distinguishing mechanism is the treatment of record order. The extracted side-stream records are reordered using keys visible to the decoder after the normalized main stream is decoded. Stable ties preserve scan order, so the decoder reconstructs the same assignment without a separately stored permutation. Our benchmark reporting follows the Large Text Compression Benchmark (LTCB), which provides a fixed full-text corpus and public ranking framework [17].

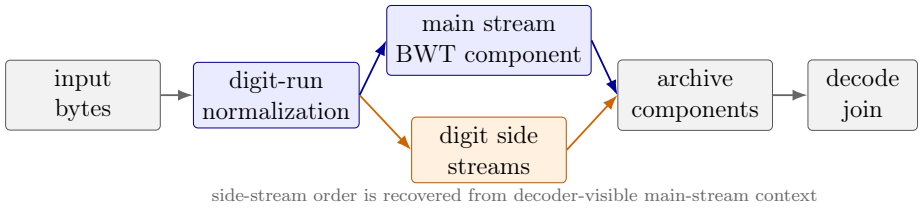


Figure 1: Overview of the STC compression and decompression pipeline. Digit runs are removed from the main stream before BWT and restored from context-conditioned side streams during decompression.

3 STC Method

3.1 Problem setting and transform boundary

STC is byte-level and schema-free: it uses only ASCII digits, not an XML grammar, token dictionary, semantic field detector, or transmitted model. It maps input x to a normalized main component and thirteen side components,

$$D(x) = (m, s_0, s_1, \ldots, s_{12}), \qquad J(D(x)) = x,$$

then applies the same fixed BWT-family coder to every component. For clarity, the paper uses *controlled result* for the same-backend ablation and *external benchmark result* for the independently maintained LTCB placement. Holding the backend fixed in the controlled result isolates whether separating textual structure from digit payloads improves compression.

3.2 Design intuition

Digit runs in structured text often vary differently from the prose or markup around them. STC separates this payload from the textual skeleton while leaving placeholders that preserve decoder-visible run positions and lengths, allowing the main BWT component to expose more regular neighborhoods. Side records are grouped by run length and ordered by normalized context recomputable from the decoded main stream, seeking both a more regular main component and better-clustered digit components without a stored permutation or schema.

3.3 Decomposition and run discovery

Let $x \in \{0, \ldots, 255\}^n$ be the input. STC maps it to a normalized main stream m and thirteen side streams $s_0, \ldots, s_{12}$, followed by a fixed component coder. Let $\mathcal{D} = \{0x30, \ldots, 0x39\}$ and choose placeholder byte $p = 0x39$. The main stream copies every non-digit byte and replaces every digit byte by p. Although p is itself ASCII `9`, it is unambiguous because all original digits, including `9`, are removed from m.

A maximal placeholder interval $[a, e)$ therefore identifies exactly one original digit run, with length $\ell = e - a$. Its bucket is $b = \min(\ell, 12)$. Define the decoder-visible

Run length	Stream	Ordering key	Packing
1	s_0	$L_3(a), N(e)$	raw
2	s_1	$L_4(a), N(e), N(e+1)$	pair
3	s_2	$N(a-1), N(e)$	pair
4	s_3, s_4	special two-stage order	int-be split
5	s_5	$L_4(a)$	int-be
6	s_6	$L_4(a), N(e)$	pair
7	s_7	$L_8(a)$	raw
8	s_8	$L_6(a)$	pair
9	s_9	$N(e), N(a-1)$	raw
10	s_{10}	$L_6(a)$	pair
11	s_{11}	$N(a-1)$	raw
≥ 12	s_{12}	$L_6(a)$	raw

Table 1: Digit-run buckets. For $\ell \geq 12$, the exact length is recovered from the placeholder run in m.

normalized-byte function

$$N(i) = \begin{cases} -1, & i < 0 \text{ or } i \geq n, \\ m_i, & \text{otherwise}, \end{cases} \qquad L_t(a) = (N(a-1), \ldots, N(a-t)).$$

The boundary value -1 is used only in ordering keys and is never serialized. Table 1 gives the bucket key and packing rule used in the released algorithm.

3.4 Decoder-reconstructible ordering

For each bucket, the encoder stably sorts run records by the key in Table 1. The decoder first reconstructs m, scans its placeholder runs, recomputes the same keys, and repeats the same stable sort. Stable ties preserve left-to-right scan order, so no explicit permutation is stored. This is the central property of STC: record order is determined entirely by information already available to both encoder and decoder.

Bucket 4 uses a two-stage split. A four-digit value is packed into two bytes. First bytes are placed in s_3 after a stable sort by $N(a-1)$. For the second stream, the original bucket-4 records are stably sorted by their already assigned first packed byte, with original left-to-right run order breaking ties, and the second bytes are placed in s_4. The decoder repeats these two deterministic stages.

3.5 Digit packing and inversion

For a run with decimal digits $d_j = c_j - 0x30$, STC uses three simple encodings. *Raw* writes each $d_j \in \{0, \ldots, 9\}$ as one byte. *Pair* stores two decimal digits as $10d_i + d_{i+1}$; for odd lengths, the first digit is marked by $100 + d_0$. *Int-be* interprets the fixed-length decimal run as

$$V = \sum_{j=0}^{\ell-1} d_j 10^{\ell-1-j}$$

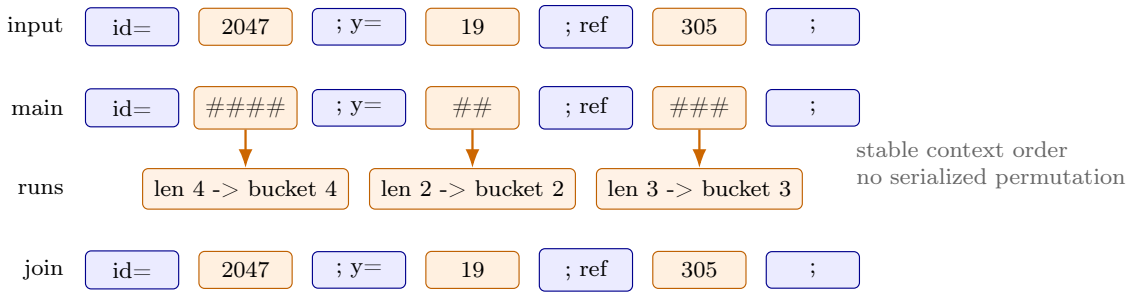


Figure 2: Worked example of reversible digit-context decomposition. Here # denotes the placeholder byte for readability. The normalized main stream preserves digit-run locations and lengths; removed payloads are assigned to length buckets and ordered using decoder-visible context.

and writes V big-endian in exactly $\lceil \ell/2 \rceil$ bytes. Decoding rejects out-of-range packed values and restores exactly ℓ digits, including leading zeros.

Proposition 1. *Given m and the thirteen side streams produced by STC, the original byte string is reconstructed uniquely.*

Proof. Scanning m recovers every digit-run position and length, hence its bucket. Every ordering key is a deterministic function of m and run position; stable sorting reproduces the encoder's tie order, including the two-stage bucket-4 rule. Raw, pair, and fixed-length int-be encodings are invertible under their validity checks. Filling each placeholder run with its decoded payload therefore restores every original digit byte, while all non-digit bytes were copied unchanged. □

The transform requires a linear scan plus stable bucket sorts, $\sum_b O(r_b \log r_b)$ for r_b records in bucket b; fixed-width context keys also permit stable radix/counting implementations. No original-order side information is required.

Algorithm provenance and fixed rule set. ZiYor was used to explore and refine candidate reversible transforms; the bucket/context table reported above is the released STC rule set evaluated in this paper. Within the same-coder ablation, this rule set is held fixed except for the explicitly disabled operation (digit separation, packing, or sorting), so rows are not independently retuned. The present paper evaluates the resulting transform rather than the general ZiYor/RSI search process.

3.6 Fixed component coder

Every nonempty component uses the same internal BWT/M03-style backend throughout the controlled ablation: a no-sentinel BWT with stored primary index, fixed root-frequency serialization, and recursive split-count coding. The normalized main stream uses the source-contained `latin205` profile and digit components use the source-contained `generic` profile; neither is trained per file or transmitted in the archive. The backend, profiles, and count tables are held constant across all ablation rows.

The outer archive contains one mode byte, fourteen compressed-component lengths, and the fourteen payloads. Lengths use little-endian base-128 varints; each nonempty

Variant	Archive bytes	Δ vs. no split
No digit split	160,017,749	0
Raw digit side stream	159,548,736	$-469,013$
Packing, sorting off	158,114,990	$-1,902,759$
Sorting, packing off	157,606,877	$-2,410,872$
Full STC	**157,388,188**	$\mathbf{-2,629,561}$

Table 2: Full-enwik9 ablation with an identical component coder.

component records its original size, BWT primary index, root-frequency information, and recursive split counts. The counted decoder package contains the decoder source and fixed tables required for decompression and invokes no external compressor. Exact bitstream details and fixed tables are provided in the public artifact, keeping the paper's experimental boundary explicit without making the backend the contribution. The recursive count coder uses deterministic boundary handling and bisection with fixed state/count tables shared by encoder and decoder; no per-file learned model is hidden outside the archive or decoder package.

4 Experimental Evaluation

4.1 Protocol

The primary dataset is enwik9, the 1,000,000,000-byte Wikipedia XML prefix used by the LTCB. Five preprocessing variants use an identical component coder, isolating the transform. Every reported archive was fully decoded; the reference SHA-256 is `159b85351e5f76e60cbe32e04c677847a9ecba3adc79addab6f4c6c7aa3744bc`, and parsed header/component lengths must equal the file-system archive size.

Verification used MinGW g++ 15.2.0 on Windows with an Intel Core i7-11700F (8 cores/16 logical processors) and 31.8 GiB RAM. Full encode/decode took 693.646/669.796 s with 12.491/12.315 GiB peak RAM; STC is a ratio-oriented research prototype.

4.2 Same-coder ablation

Table 2 gives the exact same-coder results, while Fig. 3 makes their hierarchy visually immediate. The no-split control passes the original byte stream directly to the fixed component coder. The raw-side-stream variant removes digit payloads but applies neither compact bucket packing nor context sorting. The packing/sorting-off variant uses the released bucket and packing rules while retaining scan order, whereas the sorting/packing-off variant uses the released bucket and context-ordering rules while storing digit payloads without compact packing. Thus the rows isolate separation, representation, and ordering under an otherwise unchanged backend.

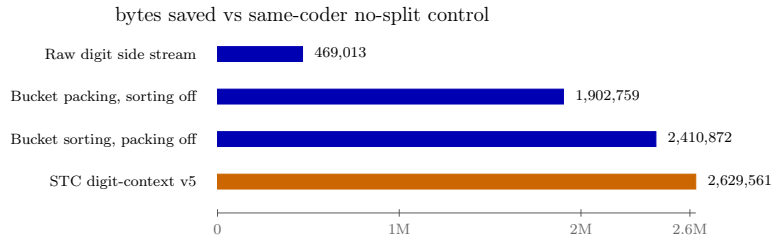


Figure 3: Full-enwik9 same-coder ablation. Bars show bytes saved relative to the no-split control; exact archive sizes are reported in Table 2.

Component	Raw bytes	Compressed bytes	Notes
m	1,000,000,000	148,105,611	Placeholder-normalized main stream
s_0	3,507,416	1,028,681	Bucket 1 stream
s_1	4,761,161	2,727,656	Bucket 2 stream
s_2	3,126,704	1,473,594	Bucket 3 stream
s_3	2,648,434	232,291	Bucket 4 first-byte stream
s_4	2,648,434	1,643,310	Bucket 4 second-byte stream
s_5	793,695	426,826	Bucket 5 stream
s_6	1,138,644	787,420	Bucket 6 stream
s_7	173,768	68,389	Bucket 7 stream
s_8	1,020,848	766,332	Bucket 8 stream
s_9	39,213	14,817	Bucket 9 stream
s_{10}	125,820	90,550	Bucket 10 stream
s_{11}	5,148	1,857	Bucket 11 stream
s_{12}	61,207	20,811	Bucket 12, length ≥ 12
Header	–	43	Mode and 14 compressed lengths
Total side	20,050,492	9,282,534	Sum of $s_0, \ldots, s_{12}$
Archive	–	157,388,188	Equals file-system byte count

Table 3: Component-level accounting. Raw side-stream sizes are pre-coding payload sizes; the main stream remains 10^9 bytes after digit replacement.

4.3 Benchmark methodology, accounting, and reproducibility

Two complementary results are reported. The *controlled result* is the same-backend ablation in Table 2; the *external benchmark result* is STC's independently maintained LTCB placement. For artifact-level reproducibility, the release also records source-accounted size: the archive satisfies 157,388,188 = 43 + 148,105,611 + 9,282,534, and adding the **183,174-byte source-contained decoder package** gives a **157,571,362-byte total**. This source accounting documents exactly what was measured locally, while the public LTCB row supplies the external ranking. Table 3 exposes the component-level archive accounting.

The normalized main stream compresses to **148,105,611 bytes** and all digit side streams to **9,282,534 bytes**. Short runs dominate the side payload, while bucket 4's two streams compress very differently under their distinct deterministic orderings, consistent with the ablation's evidence that organization—not extraction alone—matters. For local full-enwik9 profiling, full encode took **693.646 s** with **12.491 GiB** peak RAM, and full decode took **669.796 s** with **12.315 GiB** peak RAM. These measurements characterize the current ratio-oriented prototype rather than establish a speed claim.

The two benchmark views answer different questions without competing with one another. The controlled result attributes the compression gain to STC under an unchanged backend. The external benchmark result establishes practical standing: the

independently maintained LTCB leaderboard, updated September 27, 2026, lists STC at rank 37 overall and as the highest-ranked entry classified as BWT [18]. For scale, bsc-m03 0.5.5 is listed below STC among BWT-class systems; the release additionally records the corresponding local source-accounted measurements. Code, source-contained decoder material, build/decode instructions, hashes, and per-component accounting are provided for independent inspection.

5 Discussion and Limitations

The principal empirical picture is now simple: the *controlled result* isolates the effect of STC with an unchanged component coder, while the *external benchmark result* places the released system on the public LTCB leaderboard. Reproduction is defined concretely by exact input/output hashes, source-contained decoding, byte-exact archive accounting, and published build/decode commands. The artifact carries the detailed package contents and verification logs, allowing the benchmark placement and the controlled experiment to be inspected independently rather than conflated.

The controlled study is intentionally centered on full enwik9 because it is the canonical LTCB corpus and supports a direct, reproducible comparison under a fixed backend. Within that scope, the ablation cleanly separates the mechanism: extraction alone gives only a modest gain, compact packing adds more, and context-conditioned stable ordering accounts for a substantial part of the improvement, with the complete transform performing best. The irregular bucket/context choices in Table 1 are therefore best read as the specification of the released deterministic transform, not as per-file parameters: once STC is fixed, the same rules govern encoding, decoding, and every reported ablation row. Accordingly, the paper makes a deliberately benchmark-scoped claim: controlled improvement on enwik9 together with the current highest BWT-class LTCB placement. Supplementary enwik8, Calgary, Canterbury, and Silesia runs remain diagnostic only; broader frozen-rule evaluation can test how far the same transform extends beyond the benchmark without changing the claim established here.

The implementation should also be read as a compression-ratio-first research prototype. Full enwik9 processing currently requires roughly 12 GiB peak memory and about eleven minutes per direction on the reported machine, leaving substantial engineering work before production use. Bounded-memory operation, cache-aware and parallel component processing, malformed-archive/parser fuzzing, portability testing, and independent code review are natural next steps. ZiYor is important as algorithm provenance because STC emerged from its self-evolving search process rather than from a manually authored transform; nevertheless, this paper validates the resulting compression algorithm, not recursive self-improvement as a general methodology. Establishing the latter would require separate agent-level experiments, repeated discovery runs, intervention accounting, and appropriate non-RSI baselines.

6 Conclusions and Implications

STC establishes a concrete reversible way to separate variable decimal payloads from the textual structure that locates them before BWT coding. Its distinctive mechanism is not digit extraction alone, but decoder-reconstructible reordering: side-stream records are organized by context that can be regenerated from the normalized main stream, so the decoder can reproduce the same stable order without a serialized permutation. In the controlled full-enwik9 experiment, this complete transform reduces the archive from 160,017,749 to **157,388,188 bytes**, a **2,629,561-byte (1.64%)** improvement with the component coder held fixed.

The ablation gives the result its main interpretive value. A raw separated digit stream accounts for only 469,013 bytes of savings, whereas context sorting without packing reaches a 2,410,872-byte reduction and the combined transform performs best. This suggests a broader compression design principle worth testing: structured payloads can sometimes be moved into specialized streams and reorganized using side information that is already implicit in a reconstructible structural stream, avoiding the cost of an explicit position or permutation channel. Decimal runs are one concrete instance of that principle rather than an obvious endpoint.

STC also has an algorithm-discovery implication. The transform was found through ZiYor's self-evolving search process, providing a concrete algorithmic outcome whose compression behavior can be inspected independently of the system that proposed it. The immediate compression agenda is broader frozen-rule evaluation, backend-generalization tests, lower-memory engineering, robustness analysis, and continued independent reproduction. A separate research agenda is required to determine how reliably ZiYor can reproduce or extend such discoveries and which elements of its recursive self-improvement process are responsible for them. Source code, decoder sources, build instructions, verification materials, and supplementary results are available at `https://github.com/thu-nmrc/STC-for-BWT-FamilyText-Compression`.

References

[1] M. Burrows and D. J. Wheeler. A block-sorting lossless data compression algorithm. Technical Report 124, Digital Equipment Corporation, Systems Research Center, 1994.

[2] G. Manzini. An analysis of the Burrows-Wheeler transform. *Journal of the ACM*, 48(3):407–430, 2001.

[3] P. M. Fenwick. The Burrows-Wheeler transform for block sorting text compression: Principles and improvements. *The Computer Journal*, 39(9):731–740, 1996.

[4] A. Niemi and J. Teuhola. Burrows-Wheeler post-transformation with effective clustering and interpolative coding. *Software: Practice and Experience*, 50(9):1858–1874, 2020.

[5] D. Trinca. High-performance BWT-based encoders. *arXiv preprint arXiv:cs/0508086*, 2005.

[6] D. Díaz-Domínguez and G. Navarro. Efficient construction of the BWT for repetitive text using string compression. *Information and Computation*, 294:105088, 2023.

[7] I. Grebnov. bsc-m03: Experimental block sorting compressor based on M03 context aware compression algorithm, 2025. Pinned local audit commit cb2c2a296ebd3d74e19d3a98a839c8c2b3e1851b; accessed 2026-06-14.

[8] P. Skibiński, S. Grabowski, and S. Deorowicz. Revisiting dictionary-based compression. *Software: Practice and Experience*, 35(15):1455–1476, 2005.

[9] R. Wan. *Browsing and Searching Compressed Documents*. PhD thesis, Department of Computer Science and Software Engineering, University of Melbourne, 2003. December 2003.

[10] H. Liefke and D. Suciu. XMill: An efficient compressor for XML data. In *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data*, pages 153–164, 2000.

[11] P. M. Tolani and J. R. Haritsa. XGRIND: A query-friendly XML compressor. In *Proceedings of ICDE*, pages 225–234, 2002.

[12] P. Buneman, M. Grohe, and C. Koch. Path queries on compressed XML. In *Proceedings of VLDB*, pages 141–152, 2003.

[13] D. J. Abadi, S. R. Madden, and M. Ferreira. Integrating compression and execution in column-oriented database systems. In *Proceedings of SIGMOD*, pages 671–682, 2006.

[14] S. Melnik, A. Gubarev, J. J. Long, G. Romer, S. Shivakumar, M. Tolton, and T. Vassilakis. Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment*, 3(1–2):330–339, 2010.

[15] M. Żukowski, S. Héman, N. Nes, and P. Boncz. Super-scalar RAM-CPU cache compression. In *Proceedings of ICDE*, page 59, 2006.

[16] D. Lemire and L. Boytsov. Decoding billions of integers per second through vectorization. *Software: Practice and Experience*, 45(1):1–29, 2015.

[17] M. Mahoney. Rules for the large text compression benchmark, 2013. Accessed 2026-05-28.

[18] M. Mahoney. Large text compression benchmark, 2026. Leaderboard updated 2026-09-27; accessed 2026-09-30.